```
X::X() = default;                    <— Default initialization rules for a apply
```

This property is used for the atomic types (see section 5.2), which have their default constructor explicitly defaulted. Their initial value is always undefined unless either (a) they have static storage duration (and thus are statically initialized to zero), or (b) you explicitly invoke the default constructor to request zero initialization, or (c) you explicitly specify a value. Note that in the case of the atomic types, the constructor for initialization with a value is declared `constexpr` (see section A.4) in order to allow static initialization.

A.4. constexpr functions

Integer literals such as `42` are *constant expressions*. So are simple arithmetic expressions such as `23*2-4`. You can even use `const` variables of integral type that are themselves initialized with constant expressions as part of a new constant expression:

```
const int i=23;
const int two_i=i*2;
const int four=4;
const int forty_two=two_i-four;
```

Aside from using constant expressions to create variables that can be used in other constant expressions, there are a few things you can *only* do with constant expressions:

- Specify the bounds of an array:

```
int bounds=99;
int array[bounds];                   <— Error bounds is not a constant expression
const int bounds2=99;
int array2[bounds2];                 <— OK, bounds2 is a constant expression
```

- Specify the value of a nontype template parameter: